Assertions are simpler to explain than RSpec's expectations—and simplicity is a good thing—but that doesn't necessarily make one better than the other. RSpec's complexity provides a number of advantages over simple assert methods.

- Composability: *Matchers* are first-class objects that can be combined and used in flexible ways that simple assert methods can't.
- Negation: Matchers can be automatically negated by passing them to expect(object).not_to, with no need for you to write an assert_not_xyz or refute_xyz method to pair with assert_xyz.
- Readability: We've chosen to use a syntax that, when read out loud, sounds like an English description of the outcome you expect.
- More useful errors. For example, the expectation for the following collection of numbers:

  10-exploring-rspec-expectations/02/good_failure_messages.rb
  ```
  expect([13, 2, 3, 99]).to all be_odd
  ```

  ...tells you exactly which item in the collection failed:

  ```
  expected [13, 2, 3, 99] to all be odd

     object at index 1 failed to match:
        expected `2.odd?` to return true, got false
  ```

  The equivalent xUnit-style assertion, assert [13, 2, 3, 99].all?(&:odd), merely reports Expected false to be truthy.

Though we love using expectations, we will be the first to say they are not right for every project. You can easily use RSpec with a less complex library such as Minitest's assertions, as we demonstrated in *Library Configuration*, on page 161.

How Matchers Work

Earlier, we saw that matchers are objects that carry pass/fail logic. Let's take a closer look at how they work.

A matcher is a bit like a regular expression. Just as the regular expression /^\[warn\] / defines a category of strings (those beginning with [warn] followed by a space), a matcher defines a category of *objects*.

The resemblance doesn't end there. As you'll see in a moment, matchers implement one of the same protocols that regular expressions do, allowing them to be used in similar ways.